

Segment 3 (part-1)

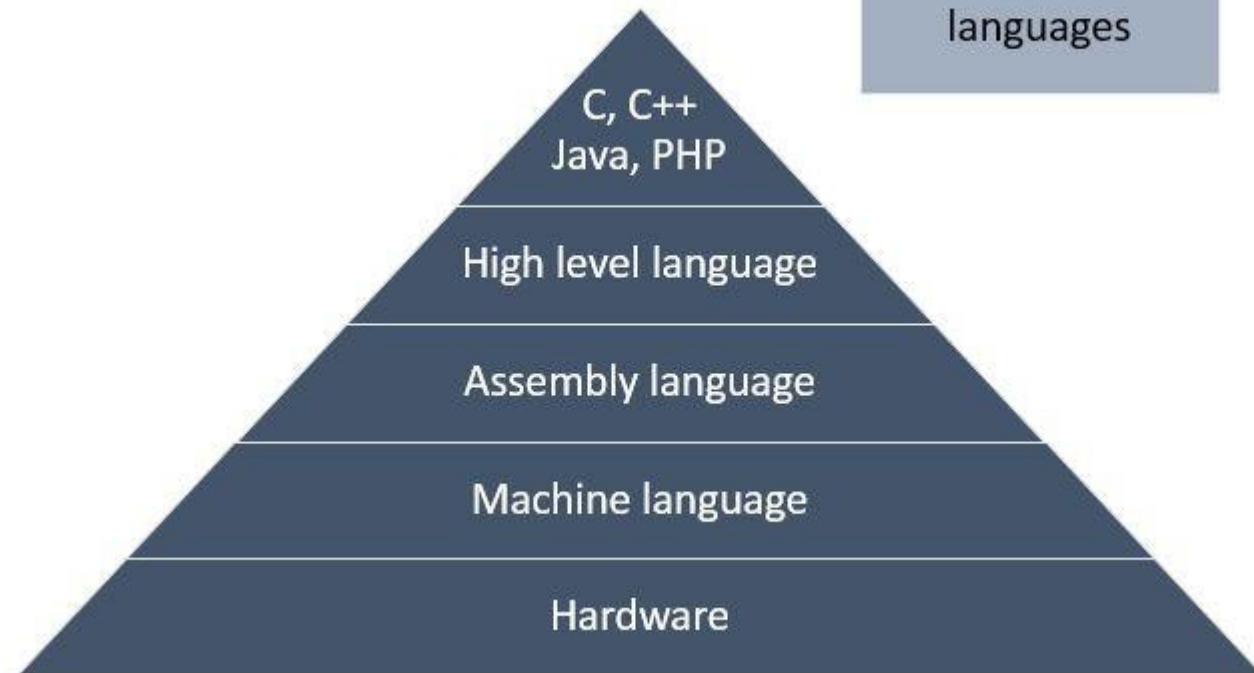
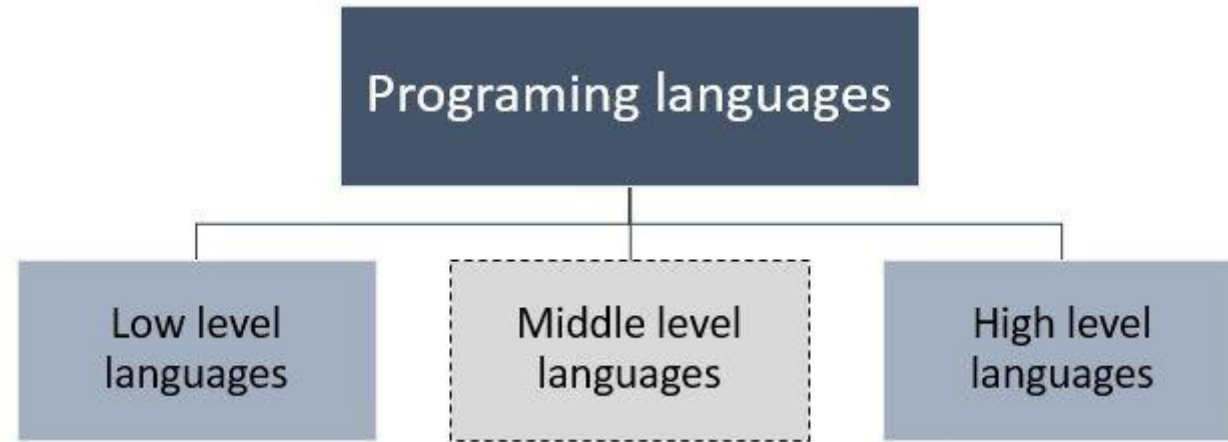
: 8086 Instruction format, Basic Concept of Assembly Language

CSE-3523: Microprocessors, Microcontrollers, and Embedded Systems

Department of CSE, IIUC

Introduction to Programming the microprocessor

Programming Language



Programming Language

Low level languages

- close to the machine level instruction set.
- provide less or no abstraction from the hardware.
- interacts directly with the registers and memory.
- Programs developed using LLL are machine dependent and are not portable.
- does not require any compiler or interpreter to translate the source to machine code.
- An assembler may translate the source code written in low level language to machine code.

Programming Language

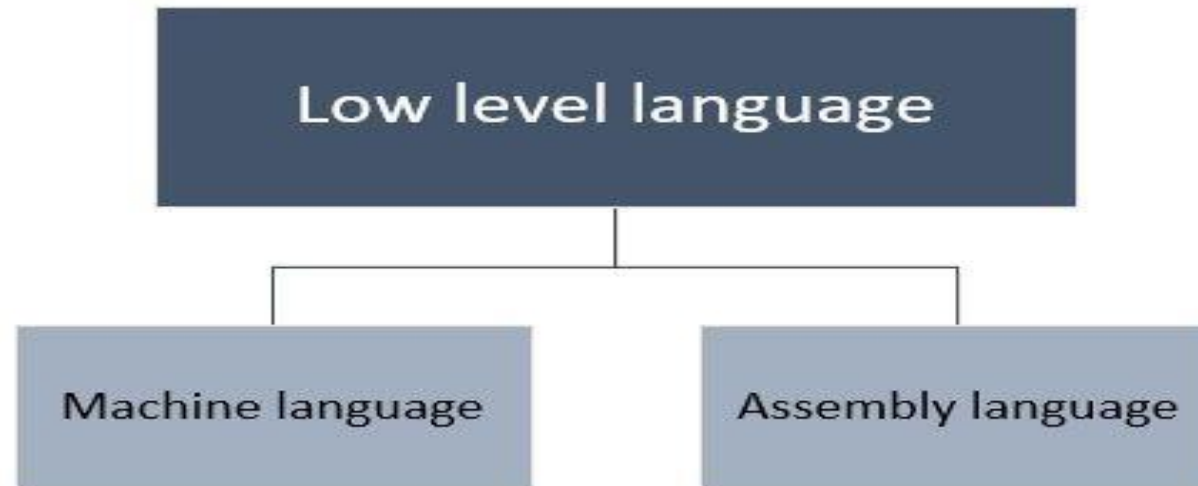
Low level languages

- Programs written in low level languages are fast and memory efficient.
- However, it is nightmare for programmers to write, debug and maintain low-level programs.
- mostly used to develop operating systems, device drivers, databases and applications that requires direct hardware access.

Programming Language

Low level languages

- Low level languages are further classified in two more categories



Programming Language

Low level languages: Machine language

- It consists set of instructions that are executed directly by the computer.
- These instructions are a sequence of binary bits.
- Instructions written in machine language are machine dependent and varies from computer to computer.

Programming Language

Low level languages: Machine language

- Example:

SUB AX, BX

= **00001011** 00000001 00100010

is an **instruction set** to subtract values of two registers **AX** and **BX**.

- A Programmer must have additional knowledge about the architecture of the particular machine, before programming in machine language.

Programming Language

Low level languages: **Assembly language**

- Assembly language is an improvement over machine language.
- Similar to machine language, assembly language also interacts directly with the hardware.
- Instead of binary sequence, assembly language uses **mnemonics** to represent **an instruction set**.

Programming Language

Low level languages: **Assembly language**

- **Mnemonics** gave relief to the programmers from remembering binary sequence for specific instructions.
- As **English words like ADD, MOV, SUB** are easy to remember, than binary sequence 10001011.
- Still have to remember various mnemonics for different computer architectures.

Programming Language

Low level languages: Assembly language

- Assembly language uses a special program called **assembler**.
 - Assembler translates mnemonics to specific machine code.
- Assembly language is **used for developing**
 - operating systems,
 - device drivers,
 - compilers and
 - other programs that requires direct hardware access.

Programming Language

Disadvantages of low level languages

- Programs developed using LLL are machine dependent and are not portable.
- difficult to develop, debug and maintain.
- Low level programs are more error prone.
- Low level programming usually results in poor programming productivity.
- Programmer must have additional knowledge of the computer architecture of particular machine, for programming in LLL.

Programming Language

Advantages of low level languages

- Programs developed using LLL are fast and memory efficient.
- Better utilization of processor and memory
- No need of any compiler or interpreters to translate the source to machine code.
 - Thus, cuts the compilation and interpretation time.
- Provide direct manipulation of computer registers and storage.
- Can directly communicate with hardware devices.

Assembly Language

Assembler

- Instruction is translated into **machine code** by an **assembler**,
 - **MASM** :
 - Microsoft Macro Assembler
 - NASM :
 - Netwide Assembler is an assembler and disassembler for the Intel x86 architecture,
 - One of the most popular assemblers for Linux
- Most assemblers are **not case sensitive**

Assembly Language Syntax

Rules

- An Assembly language program
 - consists of a **sequence of statements.**
- Assembly language instructions are so basic that,
 - Input/output are harder than High level Language
 - DOS functions are used for I/O which are fast enough to invoke

Assembly Language Syntax

Rules

- Only one statement is written per line
- Each statement consists of a sequence of ASCII characters ending with a carriage return.
- Each statement is either
 1. an **instruction** or `sub ax, bx \ c = a -b`
 2. an **assembler directive**
 - Assembler directive instructs the assembler to perform some specific task. Such as,
 - allocating memory space for a variable
 - Creating a procedure

Assembly Language Syntax

Program Statement

Both **instruction** and **assembler directive** may have up to four (4) fields.

name	operation	operand'(s)	comment
------	-----------	-------------	---------

Example of **instruction**:

START: MOV CX , 5d ; initialize counter

Example of **assembler directive** :

MAIN PROC ; creates a procedure called MAIN

- Do not have to be aligned but must appear in the above order¹⁶
- **Space or tab** separates initial fields

Assembly Language Syntax

Program Statement

- An assembly language statement contains the following fields.
 - Name Field can be used to define a symbol
 - Operation Field defines the operation code or pseudo-op
 - Operand Field specifies either the address or the data.
 - Comment Field allows the programmer to document the software

Assembly Language Syntax

Name Field

- Name field is used for:
 - instruction label: if present, a label must be followed by a colon (:)
 - procedure names
 - variable names.

Assembly Language Syntax

Name Field

- Assembler translates names into memory addresses.
- Names can be from 1 to 31 characters long: (letters, digits, and special characters: ?, ., _, \$, @, %)
- Embedded blanks are not allowed, names may not begin with a digit,
- **period** (if used) **must be the first character**

Assembly Language Syntax

Name Field

Examples:

Legal names

COUNTER1

@CHARACTER

\$500

SUM_OF_DIGITS

.TEST

DONE?

Illegal names

2ABC

TWO WORDS

A45.26

YOU&ME

Assembly Language Syntax

Operation Field

- Operation is a predefined or reserved word
 - for instruction :
 - mnemonic [symbolic operation code (opcode)]
 - for assembler directive :
 - pseudo-operation code (pseudo-op)

Assembly Language Syntax

Operation Field: for instruction

For an **instruction**

- The operation field contains a symbolic operation code (**opcode**) to describe the **operation's function**
- opcodes are **translated into machine code**.

Assembly Language Syntax

Operation Field: for instruction

- Assembly language uses **two, three or four letter mnemonics** to represent a **opcode**.
 - A mnemonic is a **symbolic name** for a single **opcode**
 - A mnemonic usually represents the **initials or shortened form** of the English word of the operation performed by the instruction.
 - **Mnemonic/opcode** for subtract is : **SUB**
 - **Mnemonic/opcode** for instruction to copy data from one location to another is : **MOV**

Assembly Language Syntax

Operation Field: for assembler directive

For an **assembler directive**

- operation field consists of a **pseudo-operation code (pseudo-op)**
- pseudo-ops are **not translated into machine code**.
- pseudo-ops simply tell assembler to do something.eg,
 - **MAIN PROC**
 - **Main proc** ; Both statements are same and are not case sensitive
 - **'Proc'** pseudo-op is used to create a procedure

Assembly Language Syntax

Operand Field: for an instruction

- Examples of instructions with different operand fields

NOP ; Instruction with **no operand** field

INC AX ; Instruction with **one operand** fields

ADD AX, 2 ; Instruction with **two operand** fields

- **If 2 operands:**

- the **first is destination** : it is register or memory location where the result is stored
- the **second is the source** operand, usually not modified by instruction

Assembly Language Syntax

Operand Field: for an assembler directive

- Usually contains **more information** about the assembler directive

Assembly Language Syntax

Comment Field

- Optional field used for documentation purposes.
- Begins with semicolon marks and assembler ignores any thing after (;)
- Importance of Comments in low-level language
- Comments in each line: Good programming practice
- Don't say obvious, say the context
 - `MOV CX,0 ;move 0 to CX`
;full line comment
 - `MOV CX,0 ;CX counts terms, initially 0`

Program Data Numbers

Examples:

number	type
1010	decimal (default)
1010B	binary
-2134D	decimal
ABFFH	illegal
1ABFFH	hex
1BHD	illegal
1BFFh	hex
1,23	illegal

Program Data

Characters

- Characters and character segments must be enclosed in single or double quotes; 'A' , "hello".
- Assembler translates characters to their ASCII code

Variables

Byte variables

- **Assembler directive** for defining **byte** variable:

Name DB initial value ;comments if any

Examples:

ALPHA DB 4 ; 1 byte variable

 ;named ALPHA is initialized to 4

Gama DB ? ; an uninitialized byte variable

Variables

Word variables (2 bytes)

Assembler directive for defining word variable:

Syntax:

Name DW initial value

Example:

WRD DW -2

- The assembler stores integers with the least significant byte in the lowest address of the memory area allocated to the integer

Example:

WD DW 1234H

low byte WD contains 34h, high byte contains 12h

Variables

Array

Assembler directive for defining 3 byte array:

```
B_ARRAY      DB      10, 25, 20
```

If array starts at offset address 0200h, it will look like this:

<u>Symbol</u>	<u>Address</u>	<u>Contents</u>
B_ARRAY	0200H	10
B_ARRAY+1	0200H+1	25
B_ARRAY+2	0200H+2	20

Variables

Array

```
W_ARRAY DW 0FFFFh, 789Ah, 0BCDEh
```

If array starts at offset address 0100h, it will look like this:

<u>Symbol</u>	<u>Address</u>	<u>Contents</u>
W_ARRAY	0100H	FFFFH
W_ARRAY+2	0102H	789AH
W_ARRAY+4	0104H	BCDEH

Variables

Array

```
INVENTORY  DW    0, 0 , 0 , 0 , 0 , 0 , 0 , 0
```

- The **TIMES** directive can also be used for multiple initializations to the same value.
- Using TIMES, the INVENTORY array can be defined as:

```
Inventory  TIMES      8      DW    0  ;
```

Variables

Character strings

Examples:

1)

LETTERS DB 'AaBCbc'

Is equivalent to

LETTERS DB 41H,61H,42H,43H,62H,63H

2)

MSG DB 'ABC',0AH,0DH,'\$'

Is equivalent to

MSG DB 41H,42H,43H,0AH,0DH,24H

Variables

Named Constant

- In an assembly language program, constants are defined through the use of the **EQU directive/pseudo-op**.
- **Syntax:**

Name **EQU** **constant**

- The EQU directive is used to assign a name to a constant.
- Use of constant names makes an assembly language easier to understand.
- **No memory is allocated** for a constant.
- The symbol on the right of **EQU** can also be a string

Variables

Named Constant

Examples:

1)

LF EQU 0AH ; LF can be used in place of 0Ah

MOV	DL	LF	} Have the same machine code
MOV	DL	0AH	

2)

PMT EQU 'TYPE YOUR NAME'

instead of

MSG DB 'TYPE YOUR NAME'

We can use

MSG DB PMT

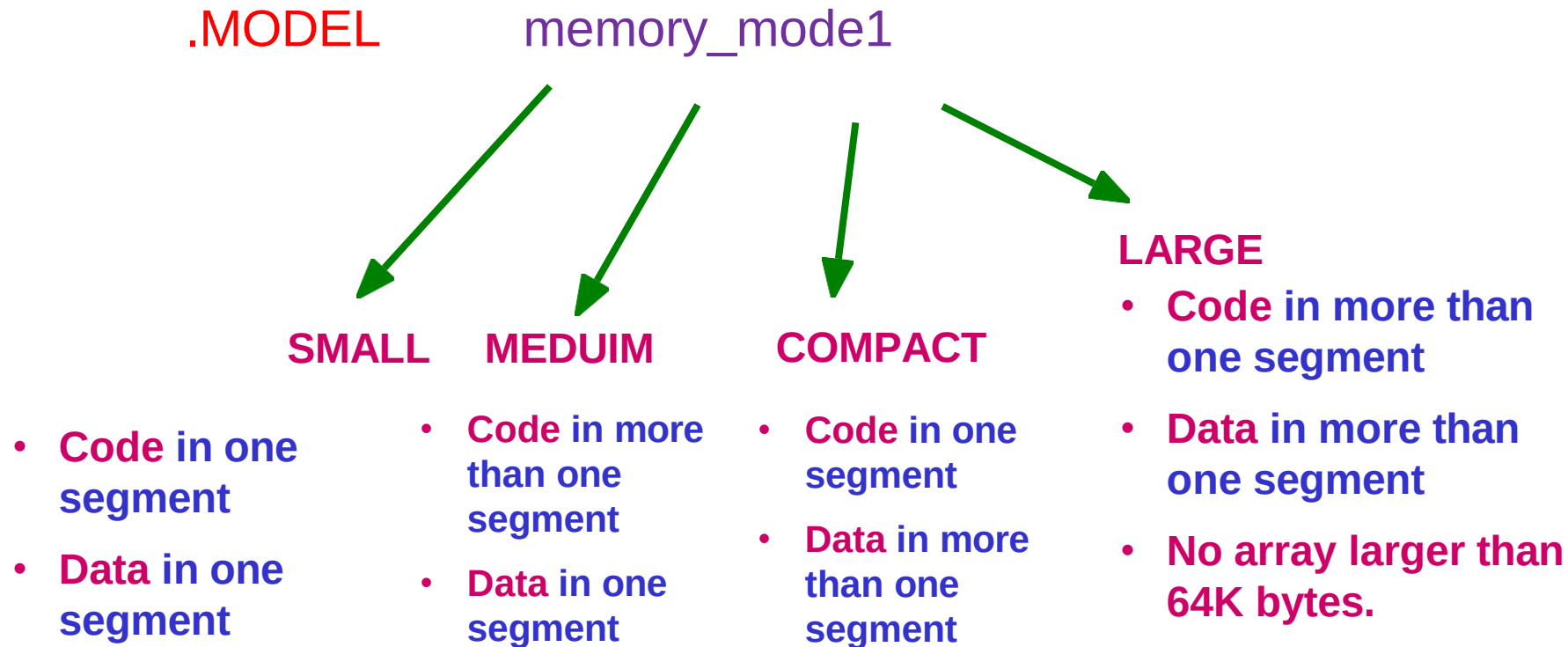
Program Structure

- Machine language programs consist of :
 1. Codes,
 2. Data, and
 3. Stack.
- Each part occupies a memory segment.
- They are structured as program segments.
- Each program segment is translated into a memory segment by the assembler.
 - We will use segment definitions introduced by MASM₃₈

Memory Models

- The size of the code & data- a program can have is determined by specifying a memory model using the `.MODEL` directive.

Memory Models



Memory Models

- Unless there is a lot of code or data, the appropriate model is **SMALL**.
- **.MODEL** directive should come before any segment definition.

Data Segment

- A program's **data segment** contains all the variable definitions.
- Constant definitions are made here as well, but they may be placed elsewhere in the program since no memory allocation is involved.
- We use the **. DATA directive** followed by **variable & constant declarations**.
- Variable addresses are computed as offsets from the start of this segment

Data Segment

Example:

.DATA

WORD1 DW 2

WORD2 DW 5

MSG DB 'This is a message '

MASK EQU 10010010B

Stack Segment

- Used to set aside storage for the stack
- Stack addresses are computed as offsets into this segment
- Use `.stack` followed by a value that indicates the size of the stack

Stack Segment

Declaration Syntax:

.STACK size



An optional number that specifies the stack area size in bytes.

Example:

STACK 100 H

- ▶ Sets aside 100h bytes for the stack area (a reasonable size for most applications) .
- ▶ If size is omitted , 1 KB is set aside for the stack area.

Code Segment

- It contains a program's instructions.

.CODE

name



Optional name for the
segment

there is no need for a name in a
SMALL program

Why??



The assembler will generate an error

Code Segment

Inside the code segment

- Inside the code segment the Instructions are organized as procedures.

- The simplest procedure definition is :

```
procedure_name  PROC  
; body of the procedure  
procedure_name  ENDP
```

- procedure_name is the name of the procedure, PROC⁷⁰ and ENDP are pseudo-op that delineate the procedure

Code Segment

Inside the code segment

Example:

```
.CODE
```

```
MAIN PROC
```

```
; main procedure body
```

```
MAIN ENDP
```

```
; other procedures go here
```


Program Structure

- A program has **always the following general structure:**

```
.model small      ;Select a memory model
.stack 100h       ;Define the stack size
.data
    ; Variable and array declarations
    ; Declare variables at this level
.code
main    proc
    ; Write the program main code at this level
main    endp
    ;Other Procedures
    ; Always organize your program into procedures
end      main      ; To mark the end of the source file
```

Procedures

- Procedure is a part of code that can be called from your program in order to make some specific task.
- Procedures make program more structural and easier to understand.
- Generally procedure returns to the same point from where it was called.

Procedures

- The syntax for procedure declaration:

procedure_name PROC

 ; here goes the code

 ; of the procedure ...

RET

procedure_name ENDP

Procedures

- `procedure_name` - is the procedure name
- **RET** instruction is used to return to operating system.
- The same instruction is used to return from procedure (actually operating system sees your program as a special procedure).
- **PROC** and **ENDP** are compiler directives, so they are not assembled into any real machine code.
- **CALL** instruction is used to call a procedure.